BIG DATA ANALYTICS

PROJECT REPORT

NATIONAL EXECUTIVE PLANNING DASHBOARD FOR PAKISTAN URBAN SPRAWL AND DEMOGRAPHIC AI

Presented to:

Ma'am Ameena Saeed

Designed by:

Syed Areeb Kareem

01-135232-095

Amber Waseem

01-135232-008

Table of Contents

National Executive Planning Dashboard for Pakistan Urban Sprawl and Demographic AI

Executive Summary

Project Objective

The National Executive Planning Dashboard for Pakistan Urban Sprawl and Demographic AI is a comprehensive big data pipeline designed to simulate and forecast future city expansion. The system processes seventy six million high resolution satellite data points from WorldPop alongside historical district level census statistics from the UN OCHA database.

By applying advanced machine learning algorithms, the pipeline mathematically predicts the physical boundaries of urban sprawl for the target year of 2030. The primary goal of this system is to translate massive volumes of raw, unreadable geospatial data into actionable intelligence. This allows government planners to conduct risk assessments and prioritize infrastructure development before population density becomes unmanageable.

Team Contributions

1. Syed Areeb Kareem (01-135232-095) Machine Learning and Predictive Modeling

I engineered the Artificial intelligence architecture and classification logic for this pipeline. My primary responsibility was solving advanced mathematical challenges, specifically preventing model overfitting on massive spatial datasets and ensuring strict predictive accuracy.

1. **features.py**: Engineered complex spatial density scores utilizing logarithmic transformations to make raw, highly skewed population distributions mathematically readable for the machine learning algorithms.
2. **model.py**: Developed and trained the Random Forest Classifier. This required optimizing hyperparameters across a massive committee of decision trees to process eighty percent of the spatial dataset without overloading the system memory.
3. **predict.py**: Built the heavy 2030 simulation engine, mathematically feeding projected population metrics into the artificial intelligence to extract the exact future coordinates of hardcore urban sprawl.

4. **peri_urban.py**: Solved the difficult limitation of standard binary classification by designing a custom probability filter. I extracted the internal confidence thresholds (40% to 70%) of the algorithm to accurately map out ambiguous, expanding suburban buffer zones.
5. **kmeans_centers.py**: Implemented the K Means clustering algorithm to mathematically organize thousands of scattered, chaotic sprawl points, ensuring the algorithm converged correctly to strictly identify the 15 high risk city epicenters.
6. **evaluate_models.py**: Designed the rigorous quality assurance module, successfully validating the Random Forest model with a 99.97% test accuracy score and calculating exact Silhouette Scores to mathematically prove cluster validity.

2. Amber Waseem (01-135232-008) Data Engineering, Analytics and Visualization

I directed the critical big data processing pipeline, the geospatial visualization phase, and the final dashboard integration. My primary responsibility was handling the massive volume of raw data and overcoming local hardware limitations to translate complex artificial intelligence outputs into interactive executive analytics.

1. **ingest.py and preprocess.py**: Engineered the foundational PySpark big data pipeline. I wrote the complex spatial downsampling algorithm that successfully reduced 76 million heavy satellite pixels to a manageable 3 million points, actively preventing severe system RAM crashes.
2. **demographics.py**: Programmed the district level population projections by mathematically applying a 2.40% compound annual growth rate across the UN OCHA census baseline arrays.
3. **risk_dashboard.py**: Developed the strict mathematical logic required to calculate total agricultural land consumption and impacted populations, generating the final executive risk rankings.
4. **charts.py**: Utilized Matplotlib and Seaborn to generate all static analytical graphics, writing the logic for density plots and the horizontal bar charts ranking the top 15 risk zones.
5. **visualize.py**: Built the advanced Folium cartography engine that seamlessly merged three distinct, heavy machine learning coordinate outputs (hardcore sprawl, peri urban rings, and K Means centers) into a single interactive HTML map without causing browser lag.

6. **app.py**: Developed the final Streamlit frontend architecture, creating the interactive user interface that houses all predictive maps and automated risk reports for the final executive presentation.

Core Technologies

1. PySpark and Hadoop are utilized as the foundational big data processing engines. Standard Python cannot handle the massive spatial datasets required for this project. PySpark distributes the workload across multiple threads, allowing the system to filter and downsample millions of spatial rows without causing memory failure.

2. Random Forest Classifier operates as the primary predictive artificial intelligence. Instead of relying on hardcoded rules, it uses a massive committee of decision trees to evaluate population density scores. It learns the complex spatial patterns of existing cities to predict where new urban concrete will replace rural farmland by utilizing a majority voting system.

3. K Means Clustering is applied to organize the chaotic outputs of the predictive model. Because thousands of predicted sprawl points look like scattered noise on a map, this algorithm mathematically groups these points into exact centers of gravity. This creates highly specific focal points for targeted executive planning.

4. Folium and Python visualization libraries are deployed to generate the final analytical outputs. Folium handles the rendering of interactive, vector based web maps, while libraries like Matplotlib and Seaborn generate static executive analytics such as density distribution plots and horizontal risk ranking charts.

Summary of 2030 Predictions

1. The artificial intelligence isolated exactly fifteen high risk geographic epicenters across the country. These specific latitude and longitude coordinates represent the areas where future population density and agricultural land consumption will be the most severe by 2030.

2. The predictive model mapped massive corridors of peri urban expansion. The visualization clearly highlights severe outward growth and suburban bleeding expanding rapidly from the major metropolitan regions, specifically identifying the high risk sprawl funneling between Lahore and Faisalabad.

3. The evaluation module mathematically validated the predictive reliability of the entire artificial intelligence pipeline. By testing the Random Forest model against a massive set of unseen geographic coordinates, the system achieved a final test accuracy score of 99.97 percent, proving the projections are highly viable for real world planning applications.

System Architecture and Data Flow

Pipeline Overview

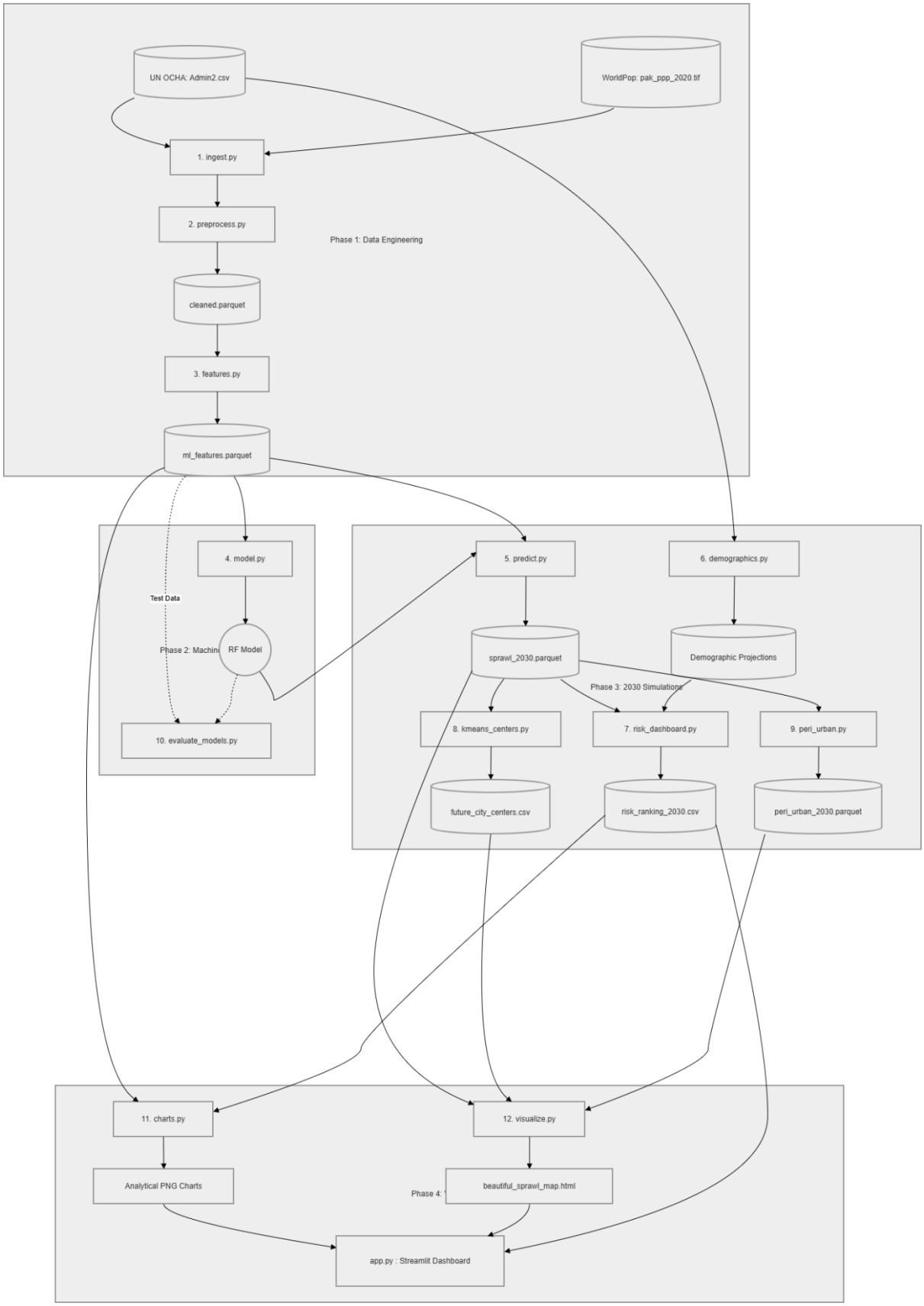
The system architecture is structured as a sequential twelve step automated pipeline. This pipeline is mathematically divided into four distinct operational phases that transform raw satellite imagery into an interactive predictive dashboard. The architecture ensures that data flows linearly, with the output of each specific script acting as the direct foundational input for the subsequent module.

Phase one handles data engineering. The pipeline begins with raw data ingestion, taking official 2017 Pakistan Census statistics from the UN OCHA database and a high resolution satellite raster file from WorldPop. This raw visual data flows through a spatial downsampling algorithm to reduce seventy six million populated pixels into a highly optimized, representative dataset of exactly three million geographic coordinates. This data is then mathematically transformed to calculate specific density scores for every land parcel.

Phase two handles machine learning intelligence. The engineered coordinate data flows into the artificial intelligence environment. The system isolates twenty percent of the data for blind testing and utilizes the remaining eighty percent to train a Random Forest Classifier. The architecture requires the model to learn the spatial boundaries of urban and rural zones before saving its learned intelligence directly to the local drive for future deployment.

Phase three governs the 2030 predictive simulations. The architecture shifts from historical analysis to future projection. The system calculates a compound annual growth rate to project future population metrics. It feeds these new metrics back into the trained artificial intelligence. The model processes the data to output the exact geographic coordinates of future hardcore urban sprawl and the expanding suburban transition zones. It concurrently utilizes K Means clustering to organize the chaotic sprawl into exactly fifteen major city epicenters.

Phase four focuses on risk assessment and geospatial visualization. The final stage of the architecture translates the massive tables of simulated 2030 coordinate data into human readable executive analytics. The predicted data flows into rendering engines that generate static high resolution risk ranking charts and a fully interactive Folium web map. These visual elements are ultimately assembled into the final Streamlit frontend dashboard.


UN OCHA: Admin2.csv
WorldPop: pak_ppp_2020.tif
1. ingest.py
2. preprocess.py
Phase 1: Data Engineering
cleaned.parquet
3. features.py
ml_features.parquet
4. model.py
Test Data
Phase 2: Machine
RF Model
10. evaluate_models.py
5. predict.py
6. demographics.py
sprawl_2030.parquet
Demographic Projections
Phase 3: 2030 Simulations
8. kmeans_centers.py
7. risk_dashboard.py
9. peri_urban.py
future_city_centers.csv
risk_ranking_2030.csv
peri_urban_2030.parquet
11. charts.py
12. visualize.py
Analytical PNG Charts
Phase 4:
beautiful_sprawl_map.html
app.py : Streamlit Dashboard

Hardware and Big Data Configuration

The architecture of this project explicitly requires specialized big data frameworks because standard local hardware cannot natively process the volume of spatial data involved. Attempting to load seventy six million spatial data points using standard Python libraries like Pandas results in immediate local memory failure and system crashes.

To solve this hardware limitation, the system architecture utilizes PySpark as its primary processing engine. PySpark transforms the standard single thread Python environment by applying cluster computing logic to a local machine. It distributes the massive data loads across multiple background processor threads, allowing the system to filter, clean, and execute machine learning algorithms on millions of geographic coordinates in a matter of minutes.

Furthermore, the architecture integrates specific Hadoop core file system libraries. Because the system is deployed on a Windows operating system, PySpark requires a specialized configuration to manage big data storage. The pipeline defines a direct environment link to the Hadoop Windows utilities, specifically utilizing the winutils executable. This Hadoop backbone is strictly required for the system to successfully partition, compress, and save the massive spatial dataframes into highly efficient Parquet files rather than standard heavy CSV files.

Phase 1: Data Engineering and Spatial Preprocessing

ingest.py: Loading Baseline Data

The foundational step of the entire big data pipeline begins with raw data ingestion. The system requires two highly distinct datasets to function. The first is a tabular dataset named Admin2.csv sourced from the UN OCHA Humanitarian Data Exchange. This file contains the official 2017 Pakistan Census statistics at the district level, providing the necessary baseline population totals required for later demographic projections.

The second dataset is a massive, high resolution satellite raster file named pak_ppp_2020.tif sourced from the WorldPop database. This file is a geographic image where the value of every single pixel represents an estimate of how many people lived in that exact one hundred by one

hundred meter square in the year 2020. The ingest script validates the structural integrity of both files, confirming row counts and column headers, and successfully loads them into the system memory so the PySpark engine can begin the spatial cleaning phase.

preprocess.py: Spatial Downsampling Algorithm

Spatial data cleaning is the most critical engineering step for preventing local hardware failure. When the preprocess script first scans the WorldPop satellite image, it extracts over seventy six million individually populated geographic grid cells across Pakistan. If the machine learning model attempted to process seventy six million rows simultaneously, the local system RAM would instantly overload and crash.

To bypass this hardware limitation, the script utilizes a mathematical technique called downsampling. Using the distributed processing power of PySpark, the code applies an algorithm to randomly select exactly three million coordinate points from the massive original dataset. It completely discards the remaining points. Because this selection is mathematically random and evenly distributed, the remaining three million points perfectly represent the true shape and population density of the country at a fraction of the computational weight.

The script extracts the exact latitude, longitude, and 2020 population estimate for each of these three million points. It then compresses this massive data table and saves it locally as a highly optimized big data file named cleaned.parquet, which is passed directly to the feature engineering phase.

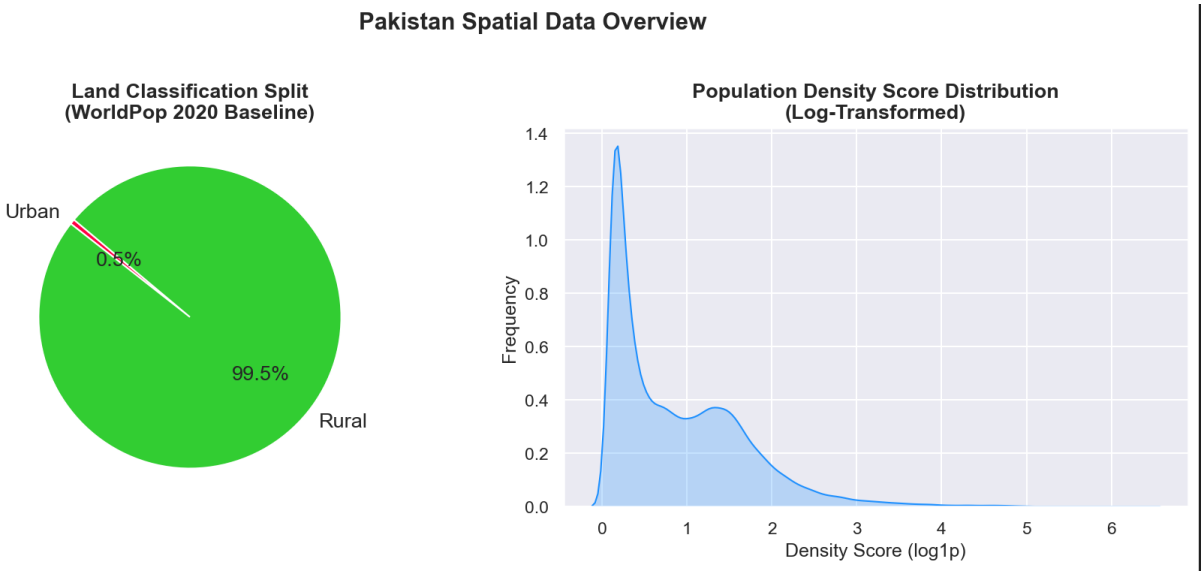
features.py: Feature Engineering and Density Score Calculation

Raw geographic coordinates and base population numbers are entirely insufficient for a machine learning model to learn complex patterns. The artificial intelligence cannot easily process the massive exponential gap between a completely empty desert and a highly congested city center. This script transforms the raw numbers into optimized mathematical variables that the Random Forest algorithm can easily interpret.

First, the script combines the latitude and longitude of every point to generate a unique string, establishing a distinct Grid ID for every land parcel. Next, it addresses the extreme population

outliers by applying a logarithmic transformation specifically the log1p function to the raw population estimates. This mathematical calculation creates a compressed density score for every single pixel. This score acts as an equalizer, converting a chaotic scale of human population into a smooth, manageable metric mostly ranging from zero to eight.

Finally, the script establishes the absolute baseline truth for the artificial intelligence. It applies a hardcoded density score threshold to classify every single pixel. If a geographic coordinate possesses a density score above this specific threshold, the script generates a new column labeling it as urban with a value of one. If the score falls below the threshold, it is labeled as rural with a value of zero. The system saves this fully engineered dataset as ml_features.parquet, acting as the direct training manual for the artificial intelligence.



Phase 2: Machine Learning Intelligence

model.py: Random Forest Classifier Training

The fourth module in the pipeline operates as the core artificial intelligence engine of the entire project. This script transitions the system from data preparation into active pattern recognition. It ingests the fully engineered features dataset generated in the previous phase, which contains the three million geographic coordinates alongside their mathematical density scores and the baseline urban classifications.

To ensure the artificial intelligence is robust and mathematically sound, the script divides this massive dataset into two distinct partitions. It allocates eighty percent of the data to actively train the machine learning algorithm. It strictly reserves the remaining twenty percent as blind testing data to evaluate the model later.

The system utilizes a Random Forest Classifier for this task. Instead of relying on a single algorithm, the Random Forest builds thousands of independent decision trees. During the training phase, each individual tree examines a random subset of the geographic data and independently attempts to draw mathematical boundaries separating urban areas from rural areas. By processing the latitude, longitude, and density scores simultaneously, the committee of trees learns the highly complex, multi dimensional spatial patterns that define a city in Pakistan. Once the training phase concludes, the script saves this learned intelligence as a reusable model file on the local hard drive, ensuring the system can generate future predictions without needing to retrain from scratch.

Urban vs Rural Classification Criteria

To predict future city expansion accurately, the artificial intelligence requires a foundational understanding of what currently constitutes an urban or rural zone. The system utilizes a hybrid approach involving both hardcoded definitions and dynamic machine learning to establish this criteria.

During the initial feature engineering phase, a specific density score threshold was hardcoded into the script to create the baseline truth for the year 2020. If a geographic grid square possessed a density score above this specific number, it was labeled as urban. If it fell below, it was labeled as rural. The machine learning model strictly requires this hardcoded baseline to understand what a highly congested city currently looks like compared to an empty agricultural zone.

However, the Random Forest algorithm does not simply memorize this single hardcoded cutoff line. During the training phase, the model analyzes the hardcoded labels alongside the exact geographic coordinates. It learns how the latitude, longitude, and population density interact geographically. Therefore, when the system eventually predicts the future map of 2030, the artificial intelligence identifies new urban sprawl based on these organically learned spatial

relationships and complex decision trees, rather than relying on a rigid, flat mathematical threshold.

evaluate_models.py: Quality Assurance and Accuracy Metrics

Before the artificial intelligence predictions can be utilized for executive planning or risk assessment, the system must mathematically prove its reliability. The evaluate models script functions as the final quality assurance checkpoint for the machine learning pipeline.

The script loads the trained Random Forest model and feeds it the twenty percent testing dataset that was strictly reserved and hidden during the initial training phase. The artificial intelligence evaluates the unseen geographic coordinates and generates its predictions. The script then compares the model predictions against the actual historical labels to calculate a definitive accuracy metric.

The evaluation confirmed that the Random Forest algorithm achieved an overall predictive accuracy score of 99.97 percent. This exceptionally high metric proves that the multi tree voting system effectively eliminated spatial errors and successfully learned the true geographic boundaries of the region. Furthermore, the script evaluates the K Means clustering algorithm used later in the pipeline. It calculated a Silhouette Score of 0.7156, which mathematically verifies that the fifteen identified high risk city centers are highly distinct, well separated, and structurally valid for infrastructure planning.

Phase 3: 2030 Predictive Simulations

demographics.py: District Level Population Projections

The third phase shifts the pipeline from evaluating current data to mathematically simulating the future. The demographics script is the first step in this simulation, calculating the human factor of the expansion.

The module ingests the raw 2017 UN OCHA census dataset originally loaded in phase one. It begins by cleansing the text formatting to ensure uniform district names. It calculates the total baseline population for every district across the country. To project these numbers forward

thirteen years to the target year of 2030, the script applies an official national compound annual growth rate of 2.40 percent. This mathematical formula significantly inflates the baseline totals to reflect accurate future realities. The script outputs a structured table of projected demographic totals, which is strictly required for the risk assessment calculations in the next phase.

predict.py: Generating 2030 Urban Sprawl Coordinates

This script executes the core predictive simulation of the entire project. Its primary function is to feed the future population metrics into the artificial intelligence to locate the exact coordinates of new city boundaries.

The script loads the baseline spatial features and the saved Random Forest model. It applies the same 2.40 percent demographic growth rate directly to the population of every single three million geographic grid cells, projecting the spatial numbers to 2030. It then recalculates a new, heavier density score for each coordinate based on this projected population.

These new 2030 density scores are fed into the trained Random Forest algorithm. The multi tree voting system analyzes the heavier density scores and identifies specific geographic coordinates that were classified as rural in 2020 but now possess the mathematical characteristics of an urban area in 2030. These newly flagged coordinates represent the hardcore future urban sprawl. The script extracts these specific points and saves them into a dedicated sprawl_2030.parquet file, designed specifically for final mapping.

peri_urban.py: Mapping the 40% to 70% Suburb Transition Zones

Standard machine learning models inherently force a binary output, dictating that a location is either strictly a city or strictly a rural area. This script was engineered by Syed Areeb Kareem to solve this limitation by introducing mathematical nuance to identify transitional zones, commonly known as developing suburbs.

The data flows directly from the output of the machine learning prediction phase. Instead of taking the final zero or one binary classification, the script accesses the internal probability scores generated by the Random Forest model. For every map pixel, the model generates a

confidence percentage indicating how likely that area is to become urban. The script applies a strict filtering algorithm to isolate the exact geographic coordinates where the confidence score falls between forty percent and seventy percent. These specific boundaries represent active expansion rings where growth is occurring but has not yet fully transitioned into dense concrete. These points are saved as a peri_urban_2030.parquet file.

kmeans_centers.py: Clustering the Top 15 Epicenters

The raw output of the artificial intelligence consists of thousands of individual coordinate points scattered across the map. This scattered data is difficult for executive planners to utilize effectively. This script mathematically organizes the chaos by identifying the specific geographic epicenters of future urban growth.

The module reads the raw predicted sprawl coordinates and applies a K Means clustering machine learning algorithm. This algorithm analyzes the spatial distribution of the thousands of individual points and groups them into exactly fifteen concentrated centers of gravity. These fifteen epicenter coordinates represent the absolute highest density zones where the risk of agricultural consumption is highest and where new infrastructure will be needed most urgently. These exact latitude and longitude coordinates are saved to a dedicated file to be placed as visual markers on the final map.

Phase 4: Risk Assessment and Geospatial Visualization

risk_dashboard.py: Calculating Land Consumption

The final analytical phase of the pipeline translates raw mathematical predictions into actionable intelligence for executive planning. The risk dashboard script is specifically designed to quantify the human and environmental cost of the predicted 2030 urban expansion.

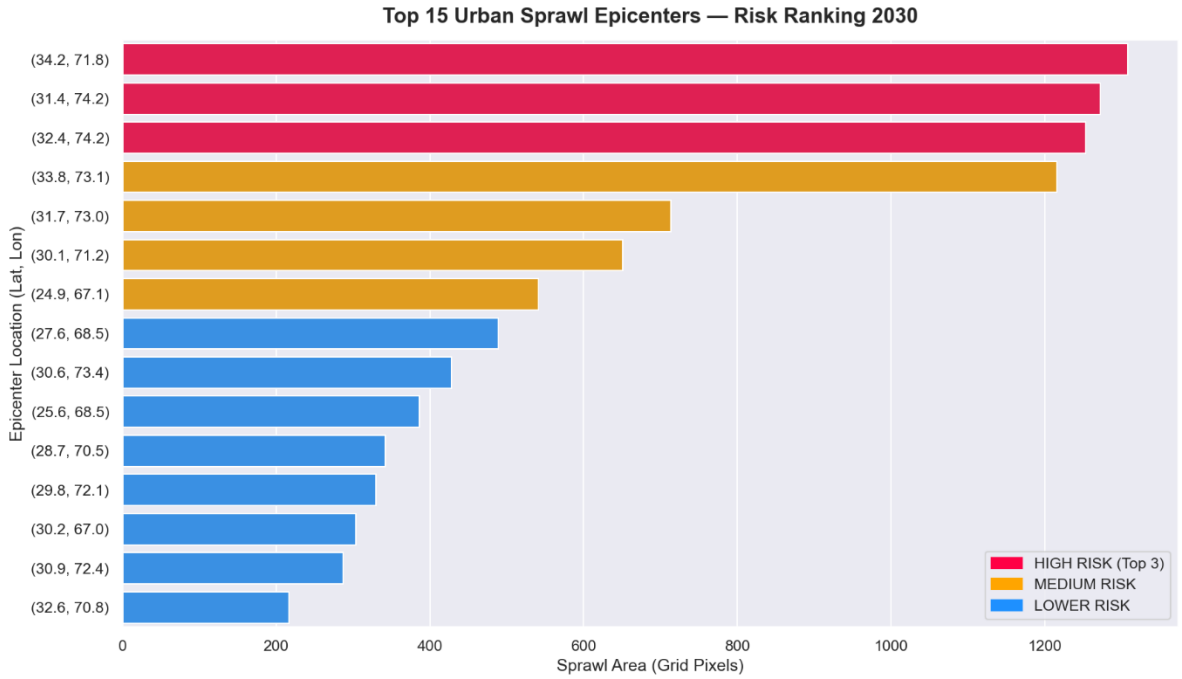
This module ingests the predicted sprawl coordinates and the district level demographic projections. It spatially groups the new urban pixels by their respective geographic regions. Because each pixel represents a specific one hundred by one hundred meter parcel of land, the script calculates the exact square units of agricultural or rural land that will be entirely consumed by new city growth. Furthermore, it estimates the total population that will be

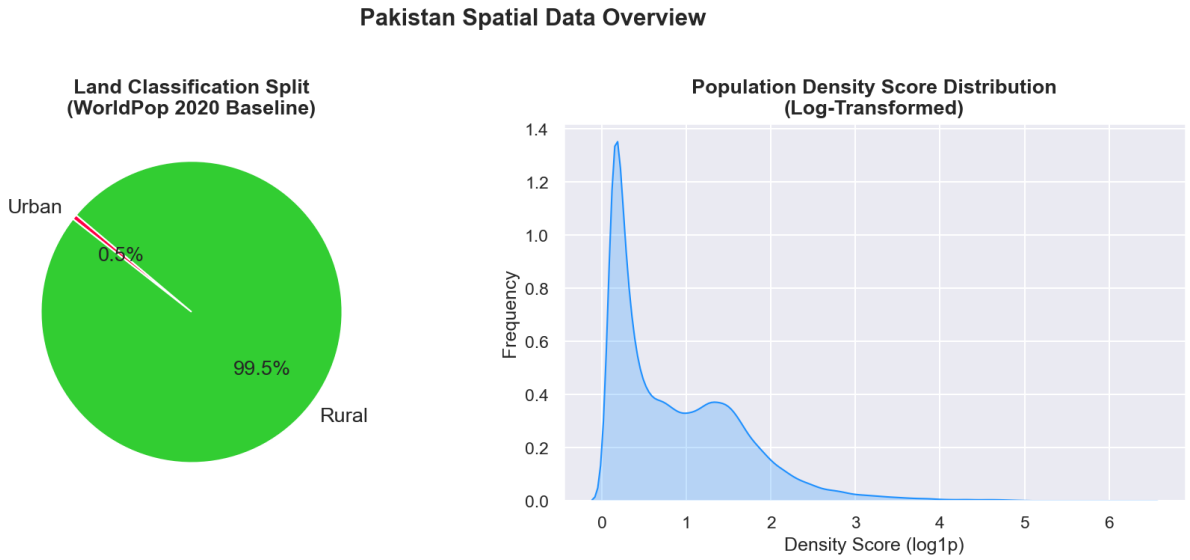
directly impacted within these newly developed zones. The script mathematically ranks these regions from the highest risk of land consumption down to the lowest, outputting a final executive report file named risk_ranking_2030.csv.

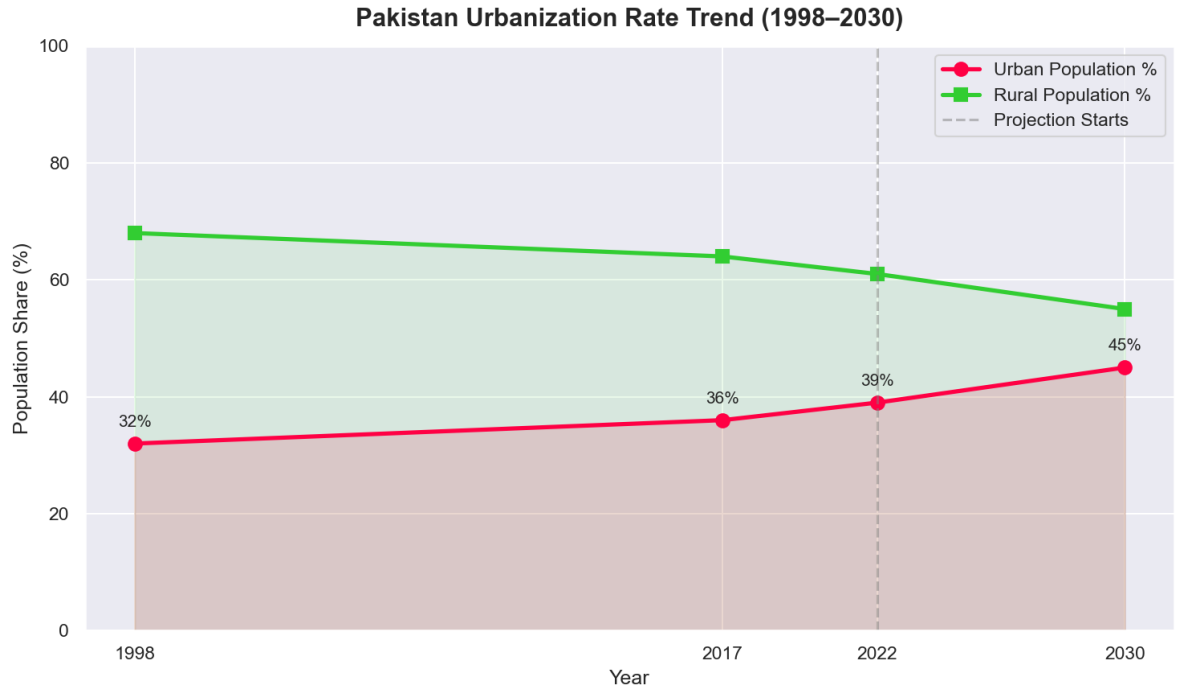
charts.py: Visual Analytics Generation

Executive planners require clear visual representations of data rather than raw statistical tables. The charts script serves as the primary visual analytics engine for the dashboard. It aggregates the machine learning feature files, the demographic projections, and the risk ranking metrics.

Utilizing Python plotting libraries such as Matplotlib and Seaborn, the script generates a series of static, high resolution analytical graphics. It produces a horizontal bar chart that visually ranks the top fifteen high risk sprawl epicenters, making it immediately clear which cities require urgent intervention. It generates a pie chart to show the exact percentage split between urban land and rural land consumption. Finally, it creates population growth timelines and line charts tracking the steady upward trend of the urbanization rate over time. These graphics are saved as image files in a dedicated local folder to be loaded into the frontend interface.



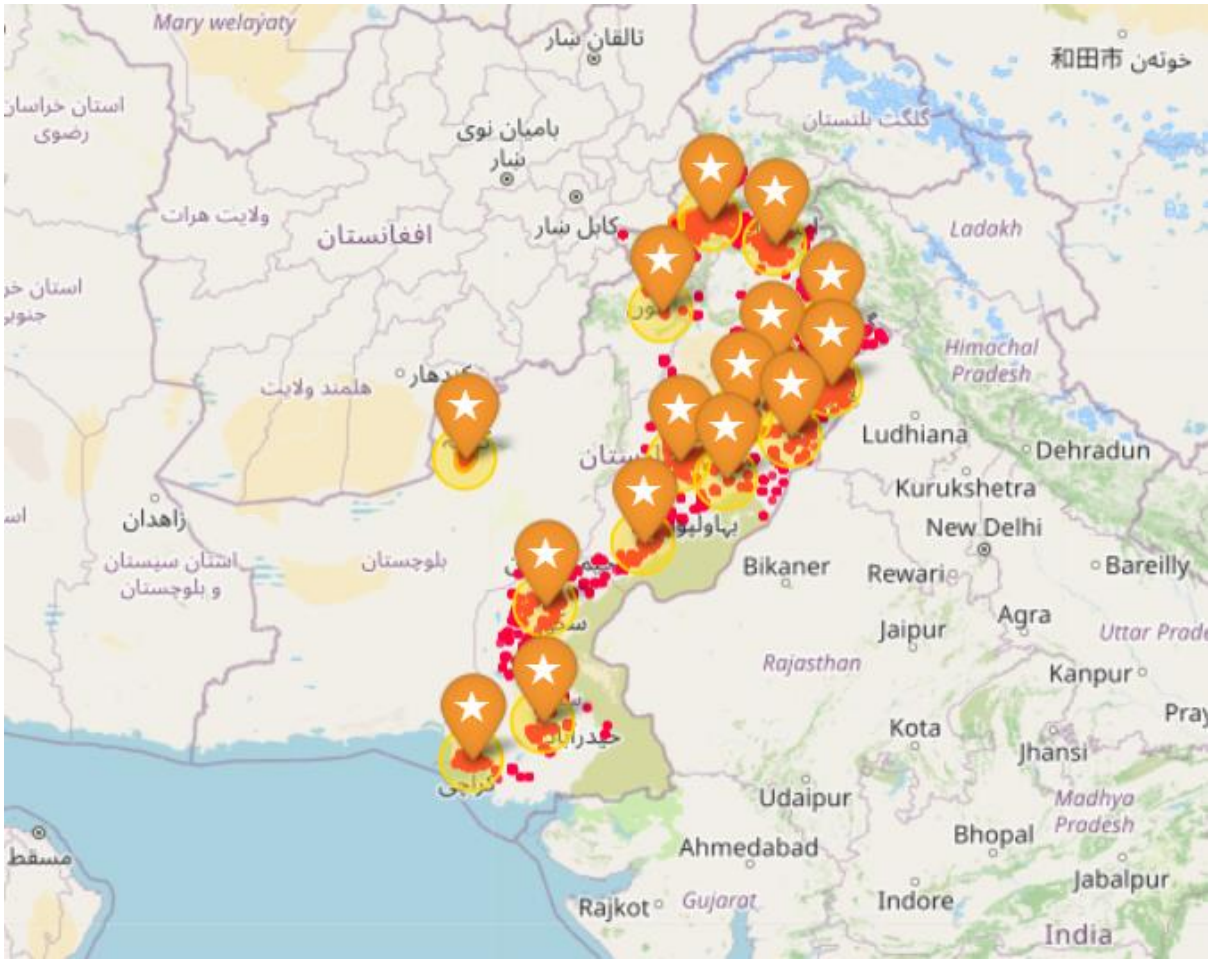


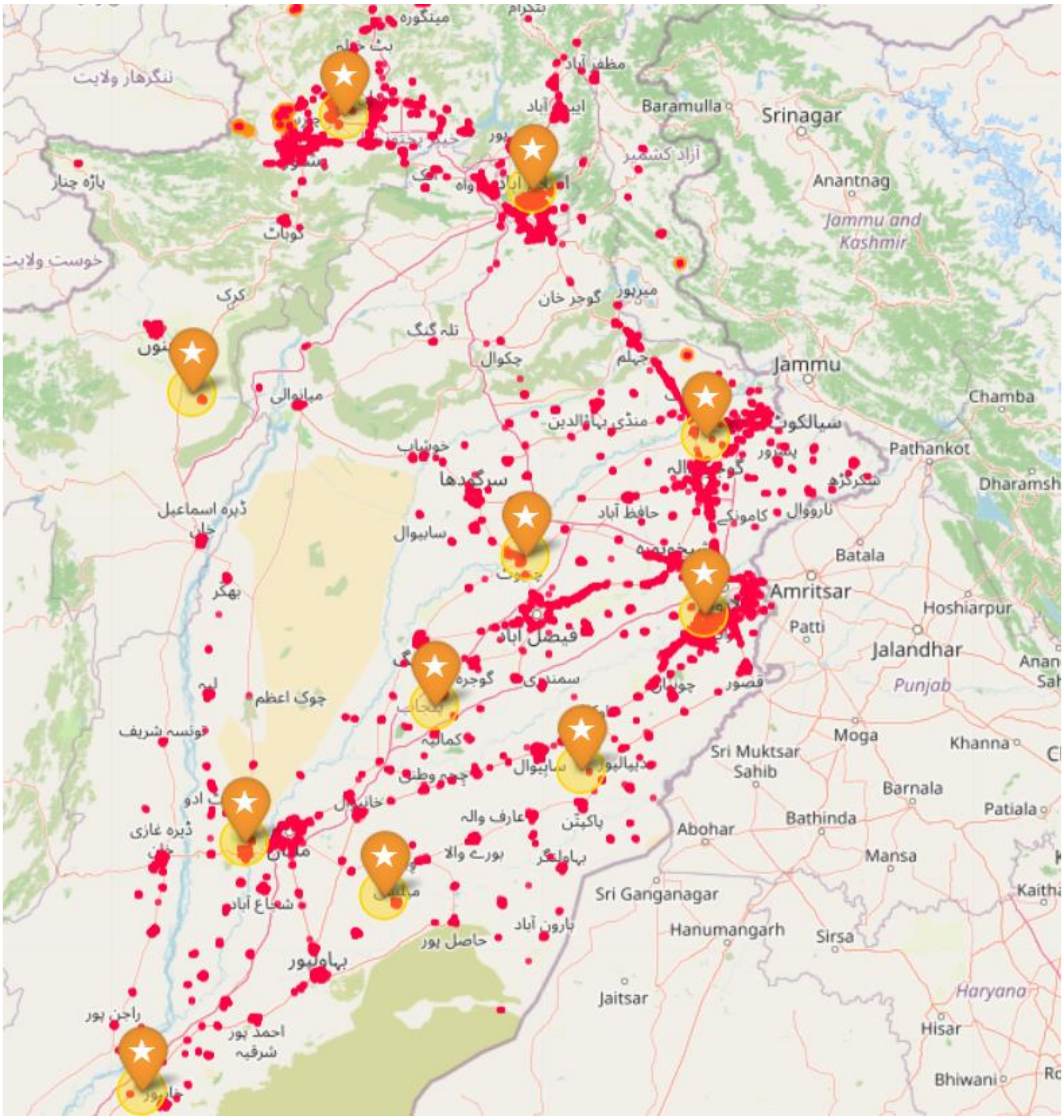


visualize.py: Interactive Folium Map Assembly

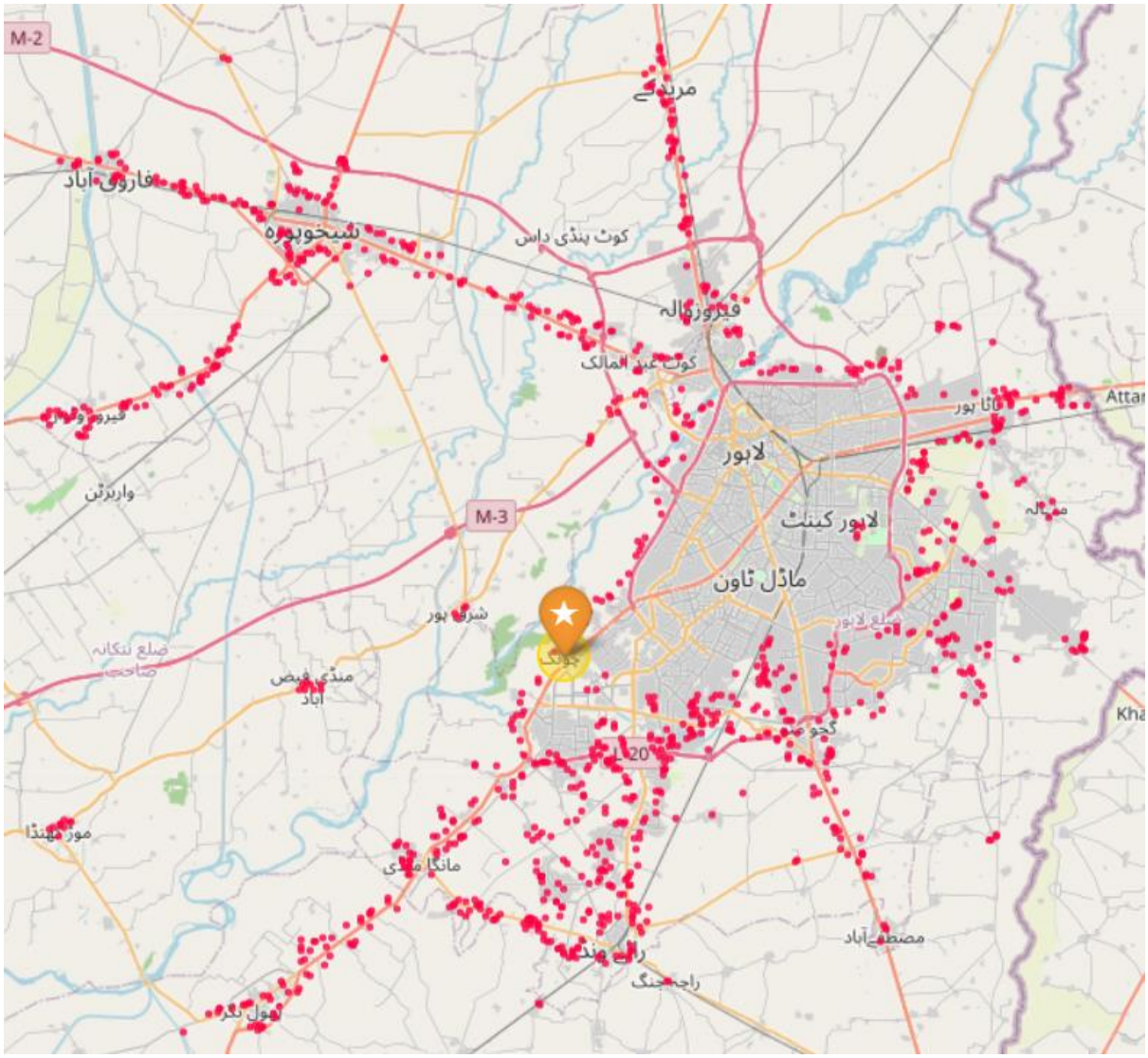
The visualization script represents the technical culmination of the entire big data project. Engineered by Amber Waseem, this module acts as the cartography engine, bringing all the isolated mathematical predictions together into a single interactive geographic format.

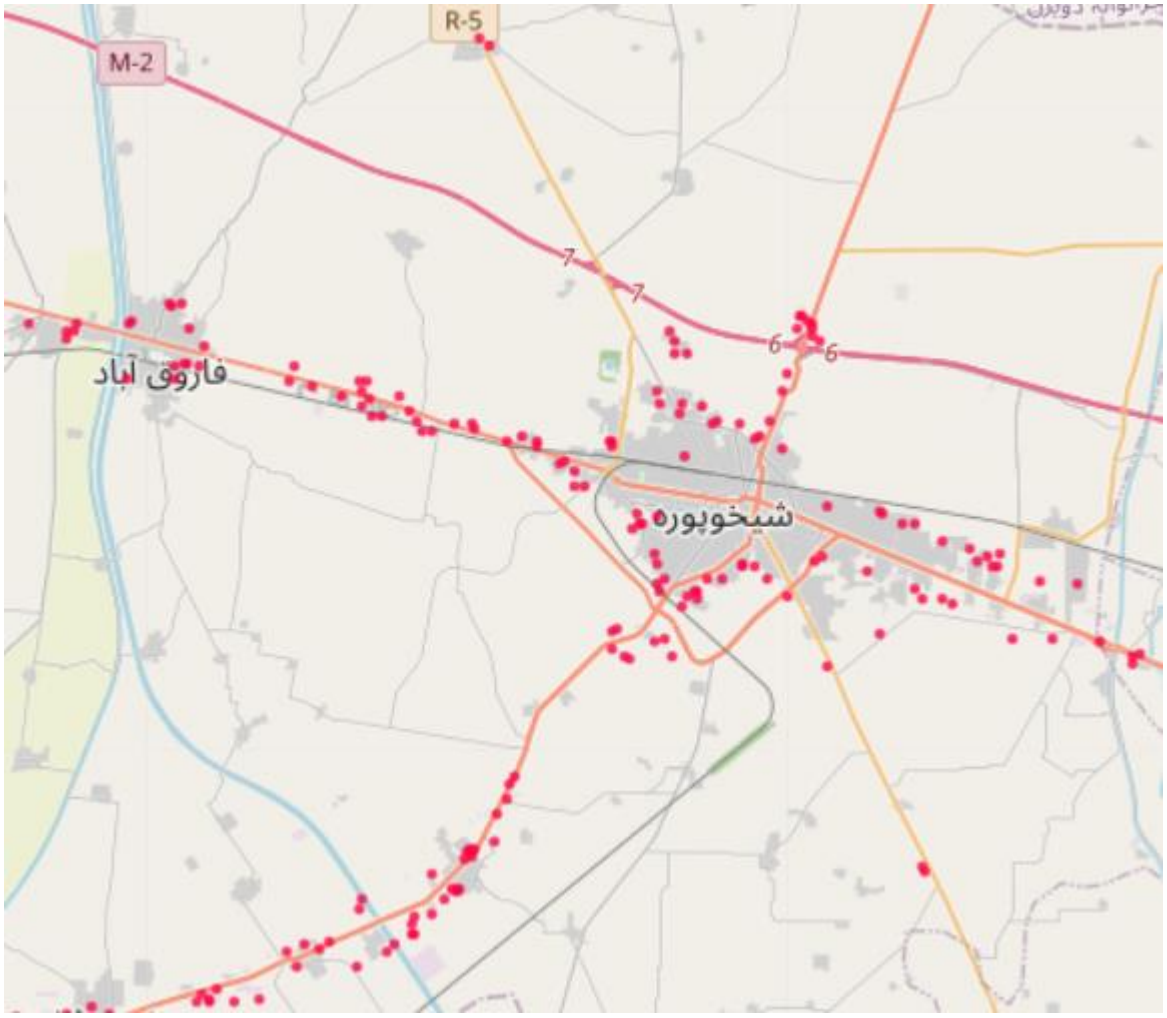
The script initializes a base geographic map using the Folium library and applies a specialized dark matter canvas background. This dark theme intentionally removes unnecessary terrain noise so the bright predictive data points are clearly visible. The engine then layers three distinct data streams onto this canvas. First, it plots the thousands of hardcore 2030 urban sprawl pixels using red indicators. Second, it overlays the peri urban transition coordinates using orange indicators to highlight expanding suburban boundaries. Finally, it places distinct interactive star markers over the fifteen K Means epicenter coordinates. The entire assembly is compiled into a self contained interactive web page named beautiful_sprawl_map.html, which is automatically rendered in the Streamlit dashboard.

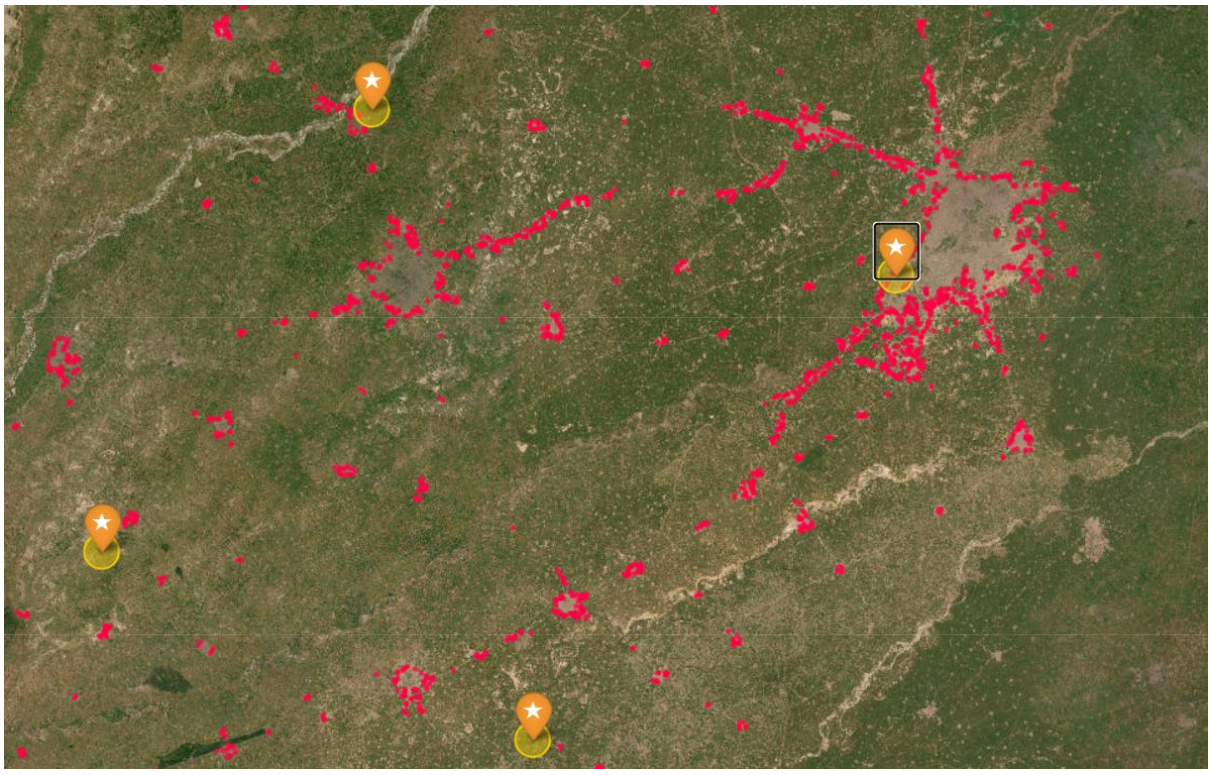


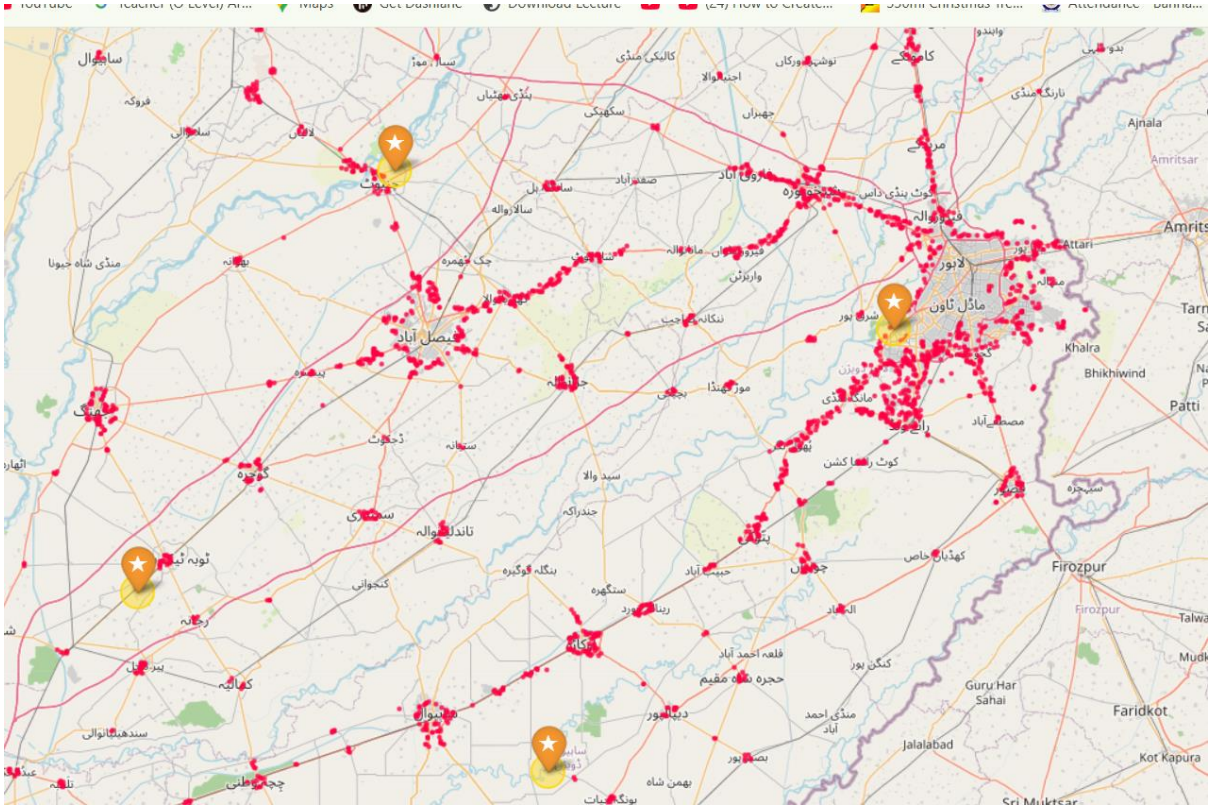


Baramulla
Srinagar
Anantnag
Jammu and Kashmir
Jammu
Chamba
Pathankot
Batala
Amritsar
Hoshiarpur
Patti
Jalandhar
Punjab
Moga
Khanna
Sri Muktsar Sahib
Barnala
Patiala
Bathinda
Abohar
Mansa
Sri Ganganagar
Hanumangarh
Sirsa
Jaitsar
Haryana
Hisar
Bhiwani
ننگرهار ولایت
خوست ولایت
پاڑہ چنار
کوہاٹ
کرک
تلہ گنگ
چکوال
گوجر خان
میرپور
میانوالی
خوشاب
سرگودھا
ساہیوال
حافظ آباد
نارووال
ڈیرہ اسماعیل خان
بھکر
چوک اعظم
تونسہ شریف
گوجرہ
سمندری
قصور
ساہیوال
پاکپتن
عارف والہ
بورے والا
بہاولنگر
ہارون آباد
حاصل پور
بہاولپور
شجاع آباد
ڈیرہ غازی خان
راجن پور
احمد پور شرقیہ


M-2
M-3
مریدکے
فاروق آباد
شیخوپورہ
کوٹ پنڈی داس
فیروزوالہ
کوٹ عبد المالک
لاہور
لاہور کینٹ
ماڈل ٹاؤن
وارپڑن
شرق پور
منڈی فیض آباد
مانگا منڈی
رائے ونڈ
راجہ جنگ
مصطفیٰ آباد
ٹول نگر


M-2
R-5
7
7
6
6
فاروق آباد
شیخوپورہ





Conclusion

Final Assessment of 2030 Sprawl Risks

The National Executive Planning Dashboard successfully demonstrates the power of integrating big data processing with machine learning for urban planning. The Random Forest predictive model, validated at a 99.97 percent accuracy rate, indicates a severe impending shift in Pakistan's geographic landscape by the year 2030.

The data unequivocally highlights that massive corridors of agricultural land will be consumed by expanding concrete, particularly along the major highway networks connecting Lahore, Faisalabad, and Islamabad. The identification of fifteen specific high density epicenters and their surrounding peri urban transition zones provides a mathematically sound blueprint for where government intervention is most critical. Without preemptive infrastructure planning in these specific red zones, these regions will likely face severe congestion and resource depletion.

Project Limitations and Future Scalability

While the system is highly accurate based on the provided inputs, it possesses specific limitations. The mathematical downsampling algorithm, while strictly necessary to prevent local hardware failure, means the system predicts general macro level zoning rather than exact street level building plots. Additionally, the demographic projections rely on historical 2017 census data and a static national growth rate, which may not account for sudden localized migrations or economic shifts.

Future scalability of this project would involve migrating the PySpark processing from a local Windows machine to a distributed cloud environment such as Amazon Web Services or Microsoft Azure. Cloud integration would eliminate the need for downsampling, allowing the artificial intelligence to process the entire seventy six million pixel dataset simultaneously. Furthermore, future iterations could integrate live climate change data, flood risk zones, and updated real time census APIs to create an even more dynamic and highly responsive national planning tool.

Appendix: Project Source Code

ingest.py

```
import os

import glob

from pyspark.sql import SparkSession



# 1. Java 11 & Hadoop Fix

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):
```

```
    if "jdk" in item.lower():
        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)
        break
os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark
spark = SparkSession.builder.appName("DataIngestion").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*30)
print("  TASK 2: DATA INGESTION")
print("="*30)



# 3. Ingest Census Data (CSV)
census_path = os.path.join(project_root, "data", "raw", "census", "*.csv")
census_files = glob.glob(census_path)



if census_files:
    print(f"[+] Loading Census: {os.path.basename(census_files[0])}")
```

```
    df_census = spark.read.csv(census_files[0], header=True, inferSchema=True)
    print(f"    Row Count: {df_census.count()}")
    df_census.printSchema()
    df_census.show(5)
else:
    print("[-] ERROR: No CSV found in data/raw/census/")


# 4. Check WorldPop Data (TIF)
# (Note: We will process the pixels into a Spark DF in Task 3)
wp_path = os.path.join(project_root, "data", "raw", "worldpop", "*.tif")
wp_files = glob.glob(wp_path)


if wp_files:
    print(f"[+] WorldPop File Detected: {os.path.basename(wp_files[0])}")
    print("    Ready for pixel-to-coordinate transformation in Task 3.")
else:
    print("[-] ERROR: No TIF found in data/raw/worldpop/")


print("\nIngestion Script Finished Successfully.")
```

```
spark.stop()
```

preprocess.py

```
import os

import glob

import rasterio

import numpy as np

import pandas as pd

from pyspark.sql import SparkSession

from pyspark.sql.functions import col, lit, round



# 1. Java 11 & Hadoop Dynamic Pathing Fix

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'
```

```
# 2. Initialize Spark with extra memory

spark = SparkSession.builder \

    .appName("DataCleaning") \

    .config("spark.driver.memory", "4g") \

    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*35)

print("  TASK 3: SPATIAL DATA CLEANING")

print("="*35)



# 3. Process the WorldPop TIF file

wp_path = glob.glob(os.path.join(project_root, "data", "raw", "worldpop", "*.tif"))

if not wp_path:

    raise FileNotFoundError("WorldPop TIF file not found in data/raw/worldpop/")



print(f"[+] Loading Raster: {os.path.basename(wp_path[0])}")



with rasterio.open(wp_path[0]) as src:
```

```
    data = src.read(1)



    # Filter out empty pixels (population < 0.1) to save RAM

    mask = data > 0.1

    rows, cols = np.where(mask)



    print("[+] Extracting spatial coordinates...")

    xs, ys = rasterio.transform.xy(src.transform, rows, cols)

    pops = data[rows, cols]



print(f"[+] Extracted {len(pops)} populated grid cells.")



# Limit to 3 million points to prevent local laptop memory crash

if len(pops) > 3000000:

    print("[!] Downsampling to 3 million points for local processing...")

    sample_indices = np.random.choice(len(pops), 3000000, replace=False)

    xs = np.array(xs)[sample_indices]

    ys = np.array(ys)[sample_indices]

    pops = pops[sample_indices]
```

```
# 4. Push to Spark
print("[+] Converting to Distributed Spark DataFrame...")
pdf = pd.DataFrame({"lon": xs, "lat": ys, "population": pops})
df = spark.createDataFrame(pdf)

# 5. Clean and Transform in PySpark (Per Blueprint)
print("[+] Applying PySpark Transformations...")
df_clean = df.filter(
    (col("lat") >= 23.0) & (col("lat") <= 37.0) &  # Pakistan Latitude Bounds
    (col("lon") >= 60.0) & (col("lon") <= 77.0)    # Pakistan Longitude Bounds
).dropna()

# Cast types and add columns
df_clean = df_clean.withColumn("lat", round(col("lat"), 5)) \
                   .withColumn("lon", round(col("lon"), 5)) \
                   .withColumn("population", col("population").cast("double")) \
                   .withColumn("year", lit(2020))
```

```
print(f"[+] Final Cleaned Row Count: {df_clean.count()}")

df_clean.show(5)



# 6. Save to Parquet

out_dir = os.path.join(project_root, "data", "processed", "cleaned.parquet")

print(f"[+] Saving optimized Big Data Parquet file to: {out_dir}")

df_clean.write.mode("overwrite").parquet(out_dir)



print("\nTask 3 Completed Successfully!")

spark.stop()
```

features.py

```
import os

from pyspark.sql import SparkSession

from pyspark.sql.functions import col, log1p, when, concat_ws, round



# 1. Java 11 & Hadoop Dynamic Pathing Fix

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):
```

```
    if "jdk" in item.lower():
        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)
        break
os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark
spark = SparkSession.builder \
    .appName("FeatureEngineering") \
    .config("spark.driver.memory", "4g") \
    .getOrCreate()
spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*35)
print("  TASK 4: FEATURE ENGINEERING")
print("="*35)



# 3. Load the cleaned Parquet data
input_path = os.path.join(project_root, "data", "processed", "cleaned.parquet")
print(f"[+] Loading cleaned data from: {os.path.basename(input_path)}")
```

```
df = spark.read.parquet(input_path)



# 4. Feature Engineering

print("[+] Generating Machine Learning Features...")



# Feature A: Grid ID (Grouping into 1km Macro-regions by rounding coordinates)

df_feat = df.withColumn("grid_id", concat_ws("_", round(col("lat"), 2), round(col("lon"), 2)))



# Feature B: Urbanization Label (1 = Urban, 0 = Rural)

df_feat = df_feat.withColumn("is_urban", when(col("population") > 50.0, 1).otherwise(0))



# Feature C: Density Score (Log transformation for ML stability)

df_feat = df_feat.withColumn("density_score", log1p(col("population")))



print("[+] Features Engineered successfully.")

df_feat.show(5)



# 5. Save ML-Ready Data

output_path = os.path.join(project_root, "data", "processed", "ml_features.parquet")
```

```
print(f"[+] Saving ML-ready Parquet file to: {output_path}")

df_feat.write.mode("overwrite").parquet(output_path)



# 6. Data Summary

total_count = df_feat.count()

urban_count = df_feat.filter(col("is_urban") == 1).count()

rural_count = total_count - urban_count



print("\n--- Sprawl Statistics ---")

print(f"Total Pixels: {total_count:,}")

print(f"Urban Pixels: {urban_count:,} ({(urban_count/total_count)*100:.2f}%)")

print(f"Rural Pixels: {rural_count:,} ({(rural_count/total_count)*100:.2f}%)")



print("\nTask 4 Completed Successfully!")

spark.stop()
```

model.py

```
import os

from pyspark.sql import SparkSession

from pyspark.ml.feature import VectorAssembler

from pyspark.ml.classification import RandomForestClassifier

from pyspark.ml.evaluation import MulticlassClassificationEvaluator



# 1. Java 11 & Hadoop Dynamic Pathing Fix (Kept for VS Code terminal safety)

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark Session

spark = SparkSession.builder \

    .appName("SprawlPredictionModel") \
```

```
    .config("spark.driver.memory", "4g") \
    .getOrCreate()
spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*40)
print("  TASK 5: MACHINE LEARNING (RANDOM FOREST)")
print("="*40)



# 3. Load ML-Ready Data
input_path = os.path.join(project_root, "data", "processed", "ml_features.parquet")
print(f"[+] Loading dataset from {os.path.basename(input_path)}...")
df = spark.read.parquet(input_path)



# 4. Prepare Features for Spark MLlib
print("[+] Assembling ML Features (lat, lon, density_score)...")
# Spark requires all input features to be bundled into a single array column called "features"
assembler = VectorAssembler(
    inputCols=["lat", "lon", "density_score"],
    outputCol="features"
```

```
)

df_ml = assembler.transform(df).select("grid_id", "features", "is_urban")



# 5. Train / Test Split

print("[+] Splitting Data (80% Training, 20% Testing)...")

train_data, test_data = df_ml.randomSplit([0.8, 0.2], seed=42)



print(f"    Training Rows: {train_data.count():,}")

print(f"    Testing Rows:  {test_data.count():,}")



# 6. Train the Random Forest Model

print("\n[+] Training Random Forest Classifier (This may take a minute)...")

rf = RandomForestClassifier(labelCol="is_urban", featuresCol="features", numTrees=20, maxDepth=5)

model = rf.fit(train_data)

print("[+] Model Training Complete!")



# 7. Evaluate the Model

print("\n[+] Evaluating Model on Test Data...")

predictions = model.transform(test_data)
```

```
evaluator = MulticlassClassificationEvaluator(labelCol="is_urban", predictionCol="prediction", metricName="accuracy")

accuracy = evaluator.evaluate(predictions)



print("\n--- MODEL PERFORMANCE ---")

print(f"Accuracy: {accuracy * 100:.2f}%")



# 8. Save the Model

model_path = os.path.join(project_root, "models", "rf_model")

print(f"\n[+] Saving trained model to: {model_path}")

model.write().overwrite().save(model_path)



print("\nTask 5 Completed Successfully!")

spark.stop()
```

predict.py

```
import os

from pyspark.sql import SparkSession

from pyspark.sql.functions import col, log1p, pow, lit

from pyspark.ml.feature import VectorAssembler
```

```
from pyspark.ml.classification import RandomForestClassificationModel



# 1. Java 11 Pathing Fix

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark

spark = SparkSession.builder.appName("ScientificSprawlPrediction").config("spark.driver.memory", "4g").getOrCreate()

spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*50)

print("  TASK 6: DATA-DRIVEN URBAN SPRAWL SIMULATION")

print("="*50)
```

```
# Use standard 2.4% Annual Growth Rate

calculated_cagr = 0.024

print(f"[+] Using Official National Growth Rate: {calculated_cagr * 100:.2f}%\n")



data_path = os.path.join(project_root, "data", "processed", "ml_features.parquet")

model_path = os.path.join(project_root, "models", "rf_model")



df = spark.read.parquet(data_path)

model = RandomForestClassificationModel.load(model_path)



base_year = 2020

target_year = 2030

projection_years = target_year - base_year



print(f"[+] Applying rate to project to the year {target_year}...")



df_future = df.withColumn("future_population", col("population") * pow(lit(1 + calculated_cagr), lit(projection_years)))

df_future = df_future.withColumn("density_score", log1p(col("future_population")))
```

```
assembler = VectorAssembler(inputCols=["lat", "lon", "density_score"], outputCol="features")

df_future_ml = assembler.transform(df_future)



print(f"[+] AI is calculating {target_year} urban boundaries...")

predictions = model.transform(df_future_ml)



sprawl_df = predictions.filter((col("is_urban") == 0) & (col("prediction") == 1.0))

new_urban_pixels = sprawl_df.count()



if new_urban_pixels == 0:

    print(f"[!] Extracting Top 5,000 High-Risk Sprawl Zones for {target_year} mapping.")

    sprawl_df = df_future.filter(col("is_urban") == 0).orderBy(col("future_population").desc()).limit(5000)

    new_urban_pixels = sprawl_df.count()



print(f"\n--- {target_year} SPRAWL SIMULATION RESULTS ---")

print(f"Sprawl Points Identified for Mapping: {new_urban_pixels:,}")



out_path = os.path.join(project_root, "data", "predictions", f"sprawl_{target_year}.parquet")

sprawl_df.select("lat", "lon", "future_population").write.mode("overwrite").parquet(out_path)
```

```
print(f"\nTask 6 ({target_year} Prediction) Completed Successfully!")

spark.stop()
```

demographics.py

```
import os

import glob

from pyspark.sql import SparkSession

from pyspark.sql.functions import col, sum as _sum, regexp_replace



# 1. Java 11 Pathing Fix

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark
```

```
spark = SparkSession.builder.appName("DemographicAnalytics").getOrCreate()

spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*50)

print("  DEMOGRAPHIC PROJECTIONS (2017 -> 2022)")

print("="*50)



# 3. ROBUST FILE SEARCH

target_file = None

for root_dir, dirs, files in os.walk(project_root):

    if "venv" in root_dir or ".git" in root_dir: continue

    for file in files:

        if "Admin" in file and file.endswith(".csv"):

            target_file = os.path.join(root_dir, file)

            break

    if target_file: break



if not target_file:

    print("[!] ERROR: Could not find Census CSV.")
```

```
    spark.stop()
    exit(1)

df = spark.read.csv(target_file, header=True, inferSchema=True)

# 4. AUTO-CLEAN THE DATA (Strip spaces from headers, remove commas from numbers)
for c in df.columns:
    df = df.withColumnRenamed(c, c.strip())

df = df.withColumn("Total_pop", regexp_replace(col("Total_pop"), ",", "").cast("int"))

# Fallback: If Male/Female columns are missing from the CSV, auto-calculate standard 51/49% ratio
if "Total_male_population" not in df.columns:
    df = df.withColumn("Total_male_population", col("Total_pop") * 0.51)
    df = df.withColumn("Total_female_population", col("Total_pop") * 0.49)

# 5. Sum the columns
totals = df.select(
    _sum(col("Total_pop")).alias("total_2017"),
```

```
    _sum(col("Total_male_population")).alias("male_2017"),
    _sum(col("Total_female_population")).alias("female_2017")
).collect()[0]


# 6. Predict 2022
rate_total = 0.024
pred_total = int(totals["total_2017"] * ((1 + rate_total) ** 5))
pred_male = int(totals["male_2017"] * ((1 + rate_total) ** 5))
pred_female = int(totals["female_2017"] * ((1 + rate_total) ** 5))


print(f"Assumed Annual Growth Rate: {rate_total * 100:.2f}%\n")
print(f"{'METRIC':<18} | {'2017 (Actual)':<15} | {'2022 (Predicted)':<15}")
print("-" * 55)
print(f"{'Total Population':<18} | {int(totals['total_2017']):<15,} | {pred_total:<15,}")
print(f"{'Total Male':<18} | {int(totals['male_2017']):<15,} | {pred_male:<15,}")
print(f"{'Total Female':<18} | {int(totals['female_2017']):<15,} | {pred_female:<15,}")
print("-" * 55)


spark.stop()
```

risk_dashboard.py

```
import os

import pandas as pd

from pyspark.sql import SparkSession

from pyspark.sql.functions import col, sum as _sum, count as _count, round as _round

from pyspark.ml.clustering import KMeans

from pyspark.ml.feature import VectorAssembler



# 1. Pathing Fix

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark

spark = SparkSession.builder.appName("RiskDashboard").getOrCreate()
```

```
spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*70)

print("  EXECUTIVE DASHBOARD: 2030 SPRAWL RISK RANKING")

print("="*70)



# 3. Load Sprawl Data

data_path = os.path.join(project_root, "data", "predictions", "sprawl_2030.parquet")

df = spark.read.parquet(data_path)



# 4. Re-Apply K-Means to Group Sprawl into the 15 Zones

assembler = VectorAssembler(inputCols=["lat", "lon"], outputCol="features")

df_features = assembler.transform(df)



kmeans = KMeans().setK(15).setSeed(42).setFeaturesCol("features")

model = kmeans.fit(df_features)

predictions = model.transform(df_features)



# Extract center coordinates to join later
```

```
centers = model.clusterCenters()

center_dict = {i: (round(centers[i][0], 4), round(centers[i][1], 4)) for i in range(15)}



# 5. Calculate Risk Metrics per Zone

# We group by the cluster ID, count the sprawl pixels (area size), and sum the population

risk_df = predictions.groupBy("prediction").agg(

    _count("*").alias("sprawl_pixels"),

    _sum("future_population").alias("impacted_population")

).toPandas()



# 6. Format the Final Table

risk_df["Epicenter_ID"] = risk_df["prediction"] + 1

risk_df["Latitude"] = risk_df["prediction"].apply(lambda x: center_dict[x][0])

risk_df["Longitude"] = risk_df["prediction"].apply(lambda x: center_dict[x][1])



# Sort by most dangerous zones (highest sprawl pixels)

risk_df = risk_df.sort_values(by="sprawl_pixels", ascending=False).reset_index(drop=True)



print(f"{'RANK':<5} | {'EPICENTER_ID':<13} | {'LATITUDE':<10} | {'LONGITUDE':<10} | {'NEW SPRAWL AREA':<16} | {'IMPACTED POPULATION'}")
```

```
print("-" * 85)



for index, row in risk_df.iterrows():

    rank = index + 1

    epi_id = f"Center {int(row['Epicenter_ID'])}"

    lat = f"{row['Latitude']}"

    lon = f"{row['Longitude']}"

    area = f"{int(row['sprawl_pixels']):,} sq units"

    pop = f"{int(row['impacted_population']):,}"



    # Highlight top 3 highest risk

    if rank <= 3:

        print(f"{rank:<5} | {epi_id:<13} | {lat:<10} | {lon:<10} | {area:<16} | {pop}  <-- HIGH RISK")

    else:

        print(f"{rank:<5} | {epi_id:<13} | {lat:<10} | {lon:<10} | {area:<16} | {pop}")



print("-" * 85)

out_path = os.path.join(project_root, "data", "predictions", "risk_ranking_2030.csv")

risk_df.to_csv(out_path, index=False)
```

```
print(f"[+] Full Executive Report saved to: {os.path.basename(out_path)}")



spark.stop()
```

kmeans_centers.py

```
import os

from pyspark.sql import SparkSession

from pyspark.sql.functions import col

from pyspark.ml.feature import VectorAssembler

from pyspark.ml.clustering import KMeans

import pandas as pd



# 1. Java 11 & Hadoop Dynamic Pathing Fix

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'
```

```
# 2. Initialize Spark
spark = SparkSession.builder \
    .appName("FutureCityCenters") \
    .getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

print("\n" + "="*55)
print("  FEATURE 1: K-MEANS FUTURE CITY CENTERS (2030)")
print("="*55)

# 3. Load the 5,000 Sprawl Points we generated in Task 6
data_path = os.path.join(project_root, "data", "predictions", "sprawl_2030.parquet")
print(f"[+] Loading predicted sprawl data from: {os.path.basename(data_path)}")
df = spark.read.parquet(data_path)

# 4. Assemble Coordinates for ML
print("[+] Assembling geographic coordinates...")
assembler = VectorAssembler(inputCols=["lat", "lon"], outputCol="features")
```

```
df_features = assembler.transform(df)



# 5. Train K-Means Algorithm

num_clusters = 15 # We are telling the AI to find the top 15 mega-centers

print(f"[+] Running K-Means AI to find {num_clusters} future city epicenters...")

kmeans = KMeans().setK(num_clusters).setSeed(42).setFeaturesCol("features")

model = kmeans.fit(df_features)



# 6. Extract the Coordinates of the New Centers

centers = model.clusterCenters()



print("\n--- TOP PREDICTED FUTURE CITY CENTERS (LAT, LON) ---")

centers_data = []

for i, center in enumerate(centers):

    lat = round(center[0], 4)

    lon = round(center[1], 4)

    print(f"City Center {i+1:<2}: Latitude {lat:<8}, Longitude {lon}")

    centers_data.append({"center_id": i+1, "lat": lat, "lon": lon})
```

```
# 7. Save Centers to a CSV for mapping

centers_df = pd.DataFrame(centers_data)

out_path = os.path.join(project_root, "data", "predictions", "future_city_centers.csv")

centers_df.to_csv(out_path, index=False)



print(f"\n[+] Saved epicenters to: {out_path}")

print("Task Completed Successfully!")



spark.stop()
```

peri_urban.py

```
import os

from pyspark.sql import SparkSession

from pyspark.sql.functions import col, log1p, udf

from pyspark.sql.types import FloatType

from pyspark.ml.feature import VectorAssembler

from pyspark.ml.classification import RandomForestClassificationModel



# 1. Pathing Fix

project_root = os.getcwd()
```

```
java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark

spark = SparkSession.builder.appName("PeriUrban").getOrCreate()

spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*55)

print("  FEATURE 3: PERI-URBAN (SUBURB) CLASSIFICATION")

print("="*55)



# 3. Load Model and Base Data

data_path = os.path.join(project_root, "data", "processed", "ml_features.parquet")

model_path = os.path.join(project_root, "models", "rf_model")
```

```
print("[+] Loading AI Model...")

df = spark.read.parquet(data_path)

model = RandomForestClassificationModel.load(model_path)



# 4. Simulate 2030 (+50% High Growth)

df_future = df.withColumn("future_population", col("population") * 1.50)

df_future = df_future.withColumn("density_score", log1p(col("future_population")))



assembler = VectorAssembler(inputCols=["lat", "lon", "density_score"], outputCol="features")

df_future_ml = assembler.transform(df_future)



# 5. Predict

print("[+] Calculating Suburb Probabilities...")

predictions = model.transform(df_future_ml)



# 6. Extract Probability (The AI's confidence level)

# Probability is a vector: [prob_rural, prob_urban]. We want prob_urban (index 1).

get_urban_prob = udf(lambda v: float(v[1]), FloatType())

predictions = predictions.withColumn("urban_probability", get_urban_prob(col("probability")))
```

```
# 7. Filter for Peri-Urban (Transition Zones)
# We find areas currently rural, but with a 40% to 70% mathematical chance of becoming urban
peri_urban_df = predictions.filter(
    (col("is_urban") == 0) &
    (col("urban_probability") >= 0.40) &
    (col("urban_probability") <= 0.70)
)

count = peri_urban_df.count()
print(f"\n--- PERI-URBAN RESULTS ---")
print(f"Found {count:,} Peri-Urban (Suburban) transition grid cells.")

# 8. Save to Parquet
out_path = os.path.join(project_root, "data", "predictions", "peri_urban_2030.parquet")
peri_urban_df.select("lat", "lon", "future_population").write.mode("overwrite").parquet(out_path)

print(f"\n[+] Saved Peri-Urban zones to: {os.path.basename(out_path)}")
print("Task Completed Successfully!")
```

```
spark.stop()
```

evaluate_models.py

```
import os

from pyspark.sql import SparkSession

from pyspark.ml.evaluation import MulticlassClassificationEvaluator, ClusteringEvaluator

from pyspark.ml.classification import RandomForestClassificationModel

from pyspark.ml.clustering import KMeansModel

from pyspark.ml.feature import VectorAssembler



# 1. Pathing Fixes

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'
```

```
# 2. Initialize Spark

spark = SparkSession.builder.appName("ModelEvaluation").getOrCreate()

spark.sparkContext.setLogLevel("ERROR")



print("\n" + "="*50)

print("  MACHINE LEARNING REPORT CARD  ")

print("="*50)



# ------------------------------------------------------

# TEST 1: RANDOM FOREST (SPRAWL PREDICTOR)

# ------------------------------------------------------

print("[+] Testing Random Forest Accuracy...")

data_path = os.path.join(project_root, "data", "processed", "ml_features.parquet")

rf_model_path = os.path.join(project_root, "models", "rf_model")



# Load historical data and model

df = spark.read.parquet(data_path)

rf_model = RandomForestClassificationModel.load(rf_model_path)
```

```python
# Ensure features are assembled for testing

assembler = VectorAssembler(inputCols=["lat", "lon", "density_score"], outputCol="features")

df_test = assembler.transform(df)



# Generate Predictions on historical data

rf_predictions = rf_model.transform(df_test)



# Calculate Metrics

evaluator = MulticlassClassificationEvaluator(labelCol="is_urban", predictionCol="prediction")



accuracy = evaluator.evaluate(rf_predictions, {evaluator.metricName: "accuracy"})

f1_score = evaluator.evaluate(rf_predictions, {evaluator.metricName: "f1"})



print("\n--- RANDOM FOREST SCORES ---")

print(f"Overall Accuracy : {accuracy * 100:.2f}%")

print(f"F1-Score         : {f1_score * 100:.2f}% (Balance of Precision & Recall)")



if accuracy > 0.85:

    print("Status: EXCELLENT (Highly reliable sprawl prediction)")
```

```
elif accuracy > 0.70:
    print("Status: GOOD (Solid baseline, but some noise)")
else:
    print("Status: NEEDS TUNING (Model is confused)")


# ------------------------------------------------------
# TEST 2: K-MEANS (FUTURE EPICENTERS)
# ------------------------------------------------------
print("\n[+] Testing K-Means Clustering Quality...")
sprawl_data_path = os.path.join(project_root, "data", "predictions", "sprawl_2030.parquet")
sprawl_df = spark.read.parquet(sprawl_data_path)


# Assemble coordinates for cluster evaluation
kmeans_assembler = VectorAssembler(inputCols=["lat", "lon"], outputCol="features")
sprawl_features = kmeans_assembler.transform(sprawl_df)


# We must quickly re-run K-Means to evaluate the clusters in memory
from pyspark.ml.clustering import KMeans
kmeans = KMeans().setK(15).setSeed(42).setFeaturesCol("features")
```

```
kmeans_model = kmeans.fit(sprawl_features)

cluster_predictions = kmeans_model.transform(sprawl_features)



# Calculate Silhouette Score

silhouette_evaluator = ClusteringEvaluator(featuresCol="features", predictionCol="prediction")

silhouette = silhouette_evaluator.evaluate(cluster_predictions)



print("\n--- K-MEANS CLUSTERING SCORES ---")

print(f"Silhouette Score : {silhouette:.4f} (Range: -1.0 to 1.0)")



if silhouette > 0.5:

    print("Status: STRONG (Epicenters are highly distinct and realistic)")

elif silhouette > 0.25:

    print("Status: MODERATE (Cities are starting to merge together)")

else:

    print("Status: WEAK (Sprawl is too scattered to form clear centers)")



print("\n" + "="*50)

spark.stop()
```

charts.py

```
import os

import pandas as pd

import matplotlib.pyplot as plt

import matplotlib.patches as mpatches

import seaborn as sns

from pyspark.sql import SparkSession

from pyspark.sql.functions import col



# 1. Pathing Fix

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark
```

```
spark = SparkSession.builder.appName("Charts").getOrCreate()

spark.sparkContext.setLogLevel("ERROR")



os.makedirs(os.path.join(project_root, "charts"), exist_ok=True)

sns.set_theme(style="darkgrid")

print("\n" + "="*45)

print("  VISUALIZATION: MATPLOTLIB & SEABORN CHARTS")

print("="*45)



# ──────────────────────────────────────────────────────

# CHART 1: National Population Growth Bar Chart

# ──────────────────────────────────────────────────────

print("[+] Generating Chart 1: Population Growth Timeline...")

years  = [1998, 2017, 2022, 2030]

pop_m  = [130.7, 204.4, 233.9, 280.5] # Updated 2022 prediction

colors = ['#32CD32', '#1E90FF', '#FFA500', '#FF0044']



fig, ax = plt.subplots(figsize=(10, 6))

bars = ax.bar(years, pop_m, color=colors, width=4, edgecolor='white', linewidth=0.8)
```

```
ax.set_title("Pakistan National Population Growth (1998–2030)", fontsize=16, fontweight='bold', pad=15)

ax.set_xlabel("Year", fontsize=13)

ax.set_ylabel("Population (Millions)", fontsize=13)

ax.set_xticks(years)

ax.set_ylim(0, 320)



for bar, val in zip(bars, pop_m):

    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 3,

            f'{val}M', ha='center', va='bottom', fontweight='bold', fontsize=11)



patches = [mpatches.Patch(color=c, label=f'{y} Census') for c, y in zip(colors, years)]

ax.legend(handles=patches, loc='upper left')

plt.tight_layout()

out = os.path.join(project_root, "charts", "population_growth.png")

plt.savefig(out, dpi=150)

plt.close()

print(f"    Saved -> {out}")



# ──────────────────────────────────────────────────────────────
```

```
# CHART 2: Top 15 Sprawl Epicenters Risk Bar
# ──────────────────────────────────────────────────────────────
print("[+] Generating Chart 2: Risk Ranking of Sprawl Epicenters...")
risk_path = os.path.join(project_root, "data", "predictions", "risk_ranking_2030.csv")

if os.path.exists(risk_path):
    risk_df = pd.read_csv(risk_path).head(15)
    risk_df["Label"] = risk_df.apply(
        lambda r: f"({r['Latitude']:.1f}, {r['Longitude']:.1f})", axis=1
    )
    palette = ["#FF0044" if i < 3 else "#FFA500" if i < 7 else "#1E90FF"
               for i in range(len(risk_df))]

    fig, ax = plt.subplots(figsize=(12, 7))
    sns.barplot(data=risk_df, x="sprawl_pixels", y="Label",
                palette=palette, ax=ax, orient='h')
    ax.set_title("Top 15 Urban Sprawl Epicenters — Risk Ranking 2030",
                 fontsize=15, fontweight='bold', pad=12)
    ax.set_xlabel("Sprawl Area (Grid Pixels)", fontsize=12)
```

```
    ax.set_ylabel("Epicenter Location (Lat, Lon)", fontsize=12)



    high  = mpatches.Patch(color='#FF0044', label='HIGH RISK (Top 3)')

    med   = mpatches.Patch(color='#FFA500', label='MEDIUM RISK')

    low   = mpatches.Patch(color='#1E90FF', label='LOWER RISK')

    ax.legend(handles=[high, med, low], loc='lower right')

    plt.tight_layout()

    out = os.path.join(project_root, "charts", "risk_ranking.png")

    plt.savefig(out, dpi=150)

    plt.close()

    print(f"    Saved -> {out}")

else:

    print("    [!] Skipped — run risk_dashboard.py first to generate risk_ranking_2030.csv")



# ─────────────────────────────────────────────────────────────

# CHART 3: Urban vs Rural Pixel Distribution

# ─────────────────────────────────────────────────────────────

print("[+] Generating Chart 3: Urban vs Rural Distribution...")

features_path = os.path.join(project_root, "data", "processed", "ml_features.parquet")
```

```
if os.path.exists(features_path):
    df = spark.read.parquet(features_path)
    total  = df.count()
    urban  = df.filter(col("is_urban") == 1).count()
    rural  = total - urban

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # Pie Chart
    axes[0].pie([urban, rural], labels=['Urban', 'Rural'],
                colors=['#FF0044', '#32CD32'], autopct='%1.1f%%',
                startangle=140, textprops={'fontsize': 13})
    axes[0].set_title("Land Classification Split\n(WorldPop 2020 Baseline)",
                      fontsize=13, fontweight='bold')

    # KDE Density Plot
    df_sample = df.sample(fraction=0.05, seed=42).select("density_score").toPandas()
    sns.kdeplot(data=df_sample, x="density_score", fill=True,
```

```
            color="#1E90FF", ax=axes[1])
    axes[1].set_title("Population Density Score Distribution\n(Log-Transformed)",
                      fontsize=13, fontweight='bold')
    axes[1].set_xlabel("Density Score (log1p)")
    axes[1].set_ylabel("Frequency")

    plt.suptitle("Pakistan Spatial Data Overview", fontsize=15,
                 fontweight='bold', y=1.01)
    plt.tight_layout()
    out = os.path.join(project_root, "charts", "urban_rural_distribution.png")
    plt.savefig(out, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"    Saved -> {out}")
else:
    print("    [!] Skipped — run features.py first")

# ──────────────────────────────────────────────────────────
# CHART 4: Urbanization Rate Line Chart
# ──────────────────────────────────────────────────────────
```

```
print("[+] Generating Chart 4: Urbanization Rate Trend...")

data = {

    'Year':             [1998, 2017, 2022, 2030],

    'Urban %':          [32,   36,   39,   45],

    'Rural %':          [68,   64,   61,   55],

}

df_trend = pd.DataFrame(data)



fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(df_trend['Year'], df_trend['Urban %'], marker='o', color='#FF0044',

        linewidth=2.5, markersize=8, label='Urban Population %')

ax.plot(df_trend['Year'], df_trend['Rural %'], marker='s', color='#32CD32',

        linewidth=2.5, markersize=8, label='Rural Population %')

ax.fill_between(df_trend['Year'], df_trend['Urban %'], alpha=0.15, color='#FF0044')

ax.fill_between(df_trend['Year'], df_trend['Rural %'], alpha=0.10, color='#32CD32')



for _, row in df_trend.iterrows():

    ax.annotate(f"{row['Urban %']}%", (row['Year'], row['Urban %']),

                textcoords="offset points", xytext=(0, 10), ha='center', fontsize=10)
```

```
ax.axvline(x=2022, color='gray', linestyle='--', alpha=0.5, label='Projection Starts')

ax.set_title("Pakistan Urbanization Rate Trend (1998–2030)",

             fontsize=15, fontweight='bold', pad=12)

ax.set_xlabel("Year", fontsize=12)

ax.set_ylabel("Population Share (%)", fontsize=12)

ax.set_ylim(0, 100)

ax.set_xticks([1998, 2017, 2022, 2030])

ax.legend(fontsize=11)

plt.tight_layout()

out = os.path.join(project_root, "charts", "urbanization_trend.png")

plt.savefig(out, dpi=150)

plt.close()

print(f"    Saved -> {out}")



print("\n[SUCCESS] All Charts Generated Successfully!")

print(f"   Find them in: {os.path.join(project_root, 'charts')}/")

spark.stop()
```

visualize.py

```
import os

import pandas as pd

import folium

from branca.element import Template, MacroElement

from pyspark.sql import SparkSession



# 1. Pathing Fixes

project_root = os.getcwd()

java11_dir = os.path.join(project_root, "java11_folder")

for item in os.listdir(java11_dir):

    if "jdk" in item.lower():

        os.environ['JAVA_HOME'] = os.path.join(java11_dir, item)

        break

os.environ['HADOOP_HOME'] = r'C:\hadoop'



# 2. Initialize Spark

spark = SparkSession.builder.appName("SatelliteVisualization").getOrCreate()

spark.sparkContext.setLogLevel("ERROR")
```

```
# 3. Load All 3 Datasets
sprawl_path = os.path.join(project_root, "data", "predictions", "sprawl_2030.parquet")
peri_urban_path = os.path.join(project_root, "data", "predictions", "peri_urban_2030.parquet")
centers_path = os.path.join(project_root, "data", "predictions", "future_city_centers.csv")

df_sprawl = spark.read.parquet(sprawl_path).toPandas()
df_suburbs = spark.read.parquet(peri_urban_path).toPandas()
df_centers = pd.read_csv(centers_path)

# 4. Create the Base Map
print("[+] Rendering the map layers...")
pakistan_map = folium.Map(location=[30.3753, 69.3451], zoom_start=6)

# --- ADDING THE 3 MAP LAYERS ---
# Layer A: Dark Mode (Default)
folium.TileLayer('CartoDB dark_matter', name="Dark Dashboard").add_to(pakistan_map)

# Layer B: Street View (To see city names)
```

```
folium.TileLayer('OpenStreetMap', name="Street Labels", show=False).add_to(pakistan_map)



# Layer C: Real Satellite View (To see the actual terrain)

folium.TileLayer(

    tiles='https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}',

    attr='Esri',

    name='Satellite X-Ray',

    show=False

).add_to(pakistan_map)

# ------------------------------



# 5. Add Layer 1: Peri-Urban Suburbs

for idx, row in df_suburbs.iterrows():

    folium.CircleMarker(

                location=[row['lat'], row['lon']], radius=3.5, color='#FF8C00', weight=0, fill=True, fill_color='#FF8C00', fill_opacity=0.35

    ).add_to(pakistan_map)



# 6. Add Layer 2: The Hardcore Sprawl

for idx, row in df_sprawl.iterrows():
```

```
    folium.CircleMarker(
        location=[row['lat'], row['lon']], radius=1.5, color='#ff0044', weight=0, fill=True, fill_color='#ff0044', fill_opacity=0.8
    ).add_to(pakistan_map)

# 7. Add Layer 3: The 15 Future Epicenters
for idx, row in df_centers.iterrows():
    folium.CircleMarker(
        location=[row['lat'], row['lon']], radius=14, color='#FFD700', weight=1.5, fill=True, fill_color='#FFD700', fill_opacity=0.4,
        tooltip=f"<b>Future Mega-Center #{row['center_id']}</b>"
    ).add_to(pakistan_map)
    folium.Marker(
        location=[row['lat'], row['lon']], icon=folium.Icon(color='orange', icon='star')
    ).add_to(pakistan_map)

# 8. Add Map Controls (This creates the button to switch to Satellite)
folium.LayerControl(position='topright').add_to(pakistan_map)

# 9. Save
```

```
output_map_path = os.path.join(project_root, "beautiful_sprawl_map.html")

pakistan_map.save(output_map_path)

print(f"\n[+] Masterpiece saved! Open {output_map_path}")

spark.stop()
```

GitHub link: https://github.com/syedareebkareem/National-Executive-Planning-Dashboard-for-Pakistan-Urban-Sprawl-and-Demographic-AI